

cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith^{1,*}, [ADDITIONAL AUTHORS TBD]¹

¹Synthesize Bio

*To whom correspondence should be addressed: max@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression (DE) analysis. Its dispersion and generalized linear model (GLM) fits operate independently by gene, but the reference implementation executes this work on the CPU. Repeated DE analyses can therefore dominate evaluations of gene-expression models.

Results: We present cuDESeq2, a PyTorch reimplementation that batches DESeq2's per-gene work on the GPU while reproducing the reference step by step. Against R DESeq2 1.52.0 the closed-form stages agree to to 8.5e-4 median relative error, over six real designs (7–300 samples, $P = 2$ –5). Stage-specific validation covered normalization, dispersion fitting, GLM fitting, significance calls, and LFC shrinkage. The standard Wald pipeline includes DESeq()'s count-outlier replacement and per-gene refit. On one A100 cuDESeq2 is 3.1–13.8 $\times$ faster end-to-end than the faster of one-worker and 12-worker R across the six designs.

Availability and implementation: gpu-deseq requires PyTorch and a CUDA GPU, and is open source under the MIT license at https://github.com/synthesizebio/gpu_deseq, with reproduction scripts in `bench/` and `benchmarks/`.

Contact: max@synthesize.bio

Supplementary information: Appended to this manuscript.

1 Introduction

DESeq2 (Love et al., 2014) is a standard method for bulk RNA-seq differential-expression analysis. A typical analysis includes 5 steps: normalization, dispersion estimation, fitting a negative-binomial GLM, significance testing, and Log fold change shrinkage estimation (Zhu et al., 2019). The dispersion, GLM and shrinkage calculations are iterative and independent by gene. DESeq2 performs these calculations on the CPU; its parallel mode distributes blocks of genes among worker [cite figure 1](#).

Historically, wall time is a secondary concern for a single DE analysis because the pipeline is only performed a handful of times per project. However, wall time becomes more important when DE must be repeated 1000s of times as part of an evaluation loop or comparison of model performance. The bioinformatics community has recently seen a surge of interest in virtual cells, foundation models for gene expression data. DE is a natural tool for evaluation of such models, and the need to repeat DE analyses in a loop has become more common. Some recent benchmarks have limited wall time by utilizing rank-based tests on normalized expression as less expensive surrogates (Roohani et al., 2025; Wei et al., 2026). A faster implementation of the DESeq2 model would permit rigorous and comprehensive reference analysis to remain in that an evaluation loop.

The established route to a faster DESeq2 thus far has been algorithmic and CPU-bound. glmGamPoi (Ahlmann-Eltze and Huber, 2020) fits the Gamma-Poisson GLM 6–13× faster than DESeq2 and edgeR, and DESeq2 can delegate its dispersion and GLM fits to it. PyDESeq2 (Muzellec et al., 2023), a Python reimplementation, improves interoperability and speed on large datasets, though by its authors’ own account it yields results “similar, but not identical” to DESeq2; GPU acceleration has meanwhile transformed adjacent single-cell workflows, where rapids-singlecell and ScaleSC report 15–20× end-to-end speedups (Hu et al., 2025), but those gains come from the dense and sparse linear-algebra core: PCA, neighbor graphs, clustering.

In this paper we present the following contributions

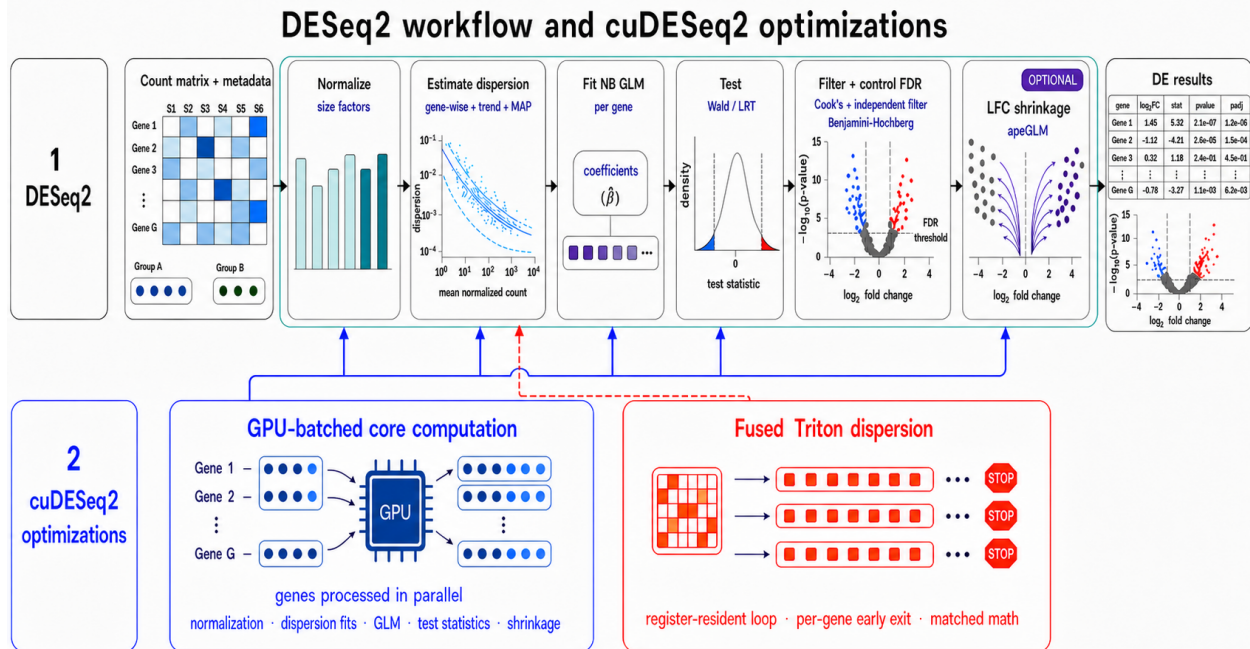


Figure 1: Overview of cuDESeq2. *Top*, the five stages of a DESeq2 run. *Bottom*, where the GPU work goes: DESeq2’s per-gene, sequential CPU work (dispersion estimation, the negative-binomial GLM, and apeGLM LFC shrinkage) is batched across all genes (*left*, solid arrows), and the dispersion fit, which is limited mainly by launch latency, has a fully fused Triton implementation (*right*, dashed arrow). A third mode uses CUDA-graph chunked replay; the three modes are compared in Table 1. Every stage is checked against R DESeq2 step by step (Sec. 3.2).

- (i) A batched GPU port of DESeq2’s `fitDisp` (Cox–Reid gradient ascent), operation-for-operation faithful to the reference and validated against R at every intermediate quantity.
- (ii) A CUDA-graph mode (chunked replay) that removes launch overhead from the dispersion loops, using a capture-safe no-pivot LU in place of `cuSOLVER`.
- 50 (iii) A fully fused Triton kernel: one program per gene runs the whole gradient-ascent loop in registers with per-gene early exit.
- (iv) A batched, on-device port of apeGLM’s own L-BFGS optimizer (limited-memory BFGS with a strong-Wolfe line search).
- (v) A per-step benchmark against R and the three GPU execution modes, with an empirical

2 Methods

2.1 Reference workflow and supported scope

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM.

For gene g with counts K_{gi} over samples $i = 1, \dots, S$, design matrix $X \in \mathbb{R}^{S \times P}$, and size
 60 factors s_i , it assumes $K_{gi} \sim \text{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$.
 The pipeline is composed of five stages.

1. **Normalization:** median-of-ratios size factors.
2. **Dispersion estimation:** gene-wise Cox-Reid MLE, a parametric mean-dispersion trend, empirical-Bayes MAP shrinkage, and an outlier rule.
- 65 3. **GLM fit / testing:** IRLS negative-binomial GLM, Wald / LRT.
4. **Significance:** Cook’s-distance filter, independent filtering, Benjamini–Hochberg adjustment.
5. **LFC shrinkage:** apeGLM (Zhu et al., 2019), a Cauchy-prior MAP estimate of the effect size.

70 This implementation supports the standard DESeq2 Wald workflow, including Cook’s-distance count replacement and refitting where applicable. It also provides lower-level functions for individual stages and LRT analyses. It does not aim to accept every R object or optional DESeq2 estimator (Sec. ??).

Stages 2, 3, and 5 contain independent per-gene iterative calculations. These are batched
 75 on the GPU.

2.1.1 Normalization

Median-of-ratios normalization is implemented with GPU tensor reductions and quantiles, including the positive-counts fallback used by DESeq2.

2.1.2 Dispersion estimation

80 Gene-wise dispersion fitting is batched across genes. The optimization methods used for this stage are described in Sec. 2.3.

2.1.3 GLM fitting and hypothesis testing

Negative-binomial IRLS fits and Wald or likelihood-ratio tests are batched over genes; the implementation details are given in Sec. 2.4.

85 2.1.4 Significance filtering and result assembly

Cook’s filtering, independent filtering, and multiple-testing adjustment are implemented as vectorized array operations. The standard wrapper also performs count replacement and refitting when DESeq2’s eligibility conditions are met.

2.1.5 LFC shrinkage

90 The apeGLM optimizer is batched across genes on the GPU; Sec. 2.5 describes the implementation.

2.2 GPU execution model

Counts, normalized counts, and per-gene state remain on the GPU. Independent gene fits are represented as batched tensors; a mask freezes genes that have converged while other genes
95 continue. Genes with zero counts in every sample are excluded from fitting and restored as missing values in the output. This is the common execution model for dispersion estimation, GLM fitting, and LFC shrinkage.

2.3 Dispersion acceleration

The dispersion stage is a batched port of DESeq2’s `fitDisp` and the `estimateDispersions` logic around it. Its gene-wise MLE maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\text{CR}}(\alpha_g) = \sum_{i=1}^S \log \text{NB}(K_{gi}; \mu_{gi}, \alpha_g) - \frac{1}{2} \log \det(X^\top W_g X), \quad (1)$$

where $W_g = \text{diag}(\mu_{gi}/(1 + \alpha_g \mu_{gi}))$. We retain DESeq2’s initialization, line search, trend fit, MAP shrinkage, and outlier rule. The optimization target is not the statistical procedure; it is the repeated gene-wise Cox–Reid evaluation in Eq. 1. The eager, CUDA-graph, and Triton implementations below use the same optimizer decisions and differ only in how those evaluations are scheduled.

Execution modes. The gradient-ascent loop is where the launch-bound overhead concentrates (Sec. 3.3), so it is the only stage with alternative execution backends. The three modes compute the same mathematics and differ in how the loop is dispatched (Table 1). They are selected per call (`fit_dispersions(..., use_cuda_graph, use_triton)`) or through the `GPU_DESEQ_ACCEL` environment variable.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \dots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P > 6$. Triton’s agreement with eager is quantified in Sec. 3.2.

CUDA-graph chunked replay. In eager mode each gradient-ascent iteration launches a handful of tiny kernels over 1–10 MB arrays, so the loop is dominated by launch latency (Sec. 3.4). Capturing the loop as a CUDA graph amortizes that cost into a single replay, but two obstacles must be overcome.

The first is early exit. A fixed full-length capture would force every gene to the maximum iteration count, which wastes the typical 15–40-iteration case. To prevent this we capture
115 a $K=10$ -iteration graph that reads and writes a set of persistent state buffers in place, and replay it in a loop with a convergence check between chunks. A chunk always runs to its boundary, so a gene that converges partway through executes further iterations.

The second is cuSOLVER. The Cox–Reid term needs $\log \det(b)$ and $\text{tr}(b^{-1}db)$ with $b = X^T W X$, but `slogdet/solve` host-synchronize and cannot be captured. Since b is
120 symmetric positive-definite for full-rank X , we replace them with an unrolled no-pivot LU (`_logdet_and_tr_binv_db`) returning both quantities from elementwise and indexing operations alone, matching the `linalg` path to $\sim 1\text{e-}15$ for $P = 2 \dots 6$. The eager path uses the same routine, which is what makes the two bit-identical.

Before capture the chunk is run three times on a side stream, warming the caching
125 allocator and any lazy kernel initialization. Captured graphs are cached on G , S , P , the chunk length K , whether a prior term is enabled, the dtype and the device. Only *whether* there is a prior is selected on the device, because the prior variance lives in the buffer set as a scalar tensor the replay reads, so one graph serves datasets whose prior variances differ.

Triton full-loop fusion. The Triton mode fuses a gene’s entire fit into a single program,
130 launched once per gene (Fig. 2). Its counts, μ and design columns are loaded once into registers, and the gradient ascent then runs register-resident: the Cox–Reid $P \times P$ solve (closed-form 2×2 for $P=2$, an unrolled no-pivot LU above that), the NB log-likelihood and $\log \alpha$ -gradient (`lgamma` and `log1p` from `libdevice`, `digamma` hand-implemented to match ATen’s `calc_digamma` through a recurrence to $x \geq 10$ followed by the Bernoulli asymptotic
135 series), and the Armijo line search. A per-gene `while` loop lets each gene exit as soon as it converges, so the kernel does not run every gene to the batch maximum. The kernel is fp64 and covers $P \in \{2, \dots, 6\}$; `fit_dispersions` gates on the covered range and routes wider designs to the eager path, while the kernel’s own launcher raises instead of fitting a

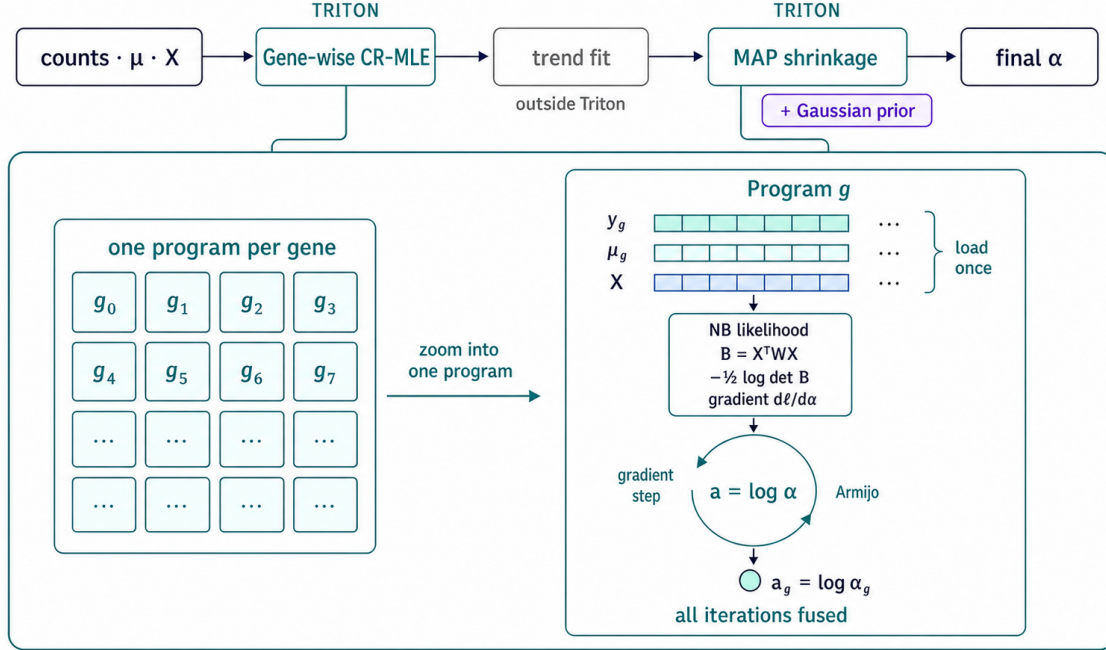


Figure 2: The fused Triton dispersion kernel. One kernel handles both gene-wise fits of the dispersion pipeline: the Cox–Reid MLE, and the MAP shrinkage with a Gaussian prior that follows the trend fit (the trend fit itself runs outside Triton). One Triton program is launched per gene; it loads y_g , μ_g and the design X once, then runs every iteration of the gradient ascent register-resident, with the NB log-likelihood, the Cox–Reid term $-\frac{1}{2} \log \det \mathbf{B}$ ($\mathbf{B} = \mathbf{X}^T \mathbf{W} \mathbf{X}$), the $\log \alpha$ -gradient and the Armijo line search all fused into the loop body. Only the converged $a_g = \log \alpha_g$ is written back to global memory. Supplementary Algorithms S1–S2 give the pseudocode.

truncated model, since it reads a fixed number of design columns. At the gene-wise step,
 140 non-converged and high-dispersion genes get the same noIncrease revert and grid fallback as
 the eager path, computed eagerly after the kernel returns; at the MAP step neither applies,
 in either mode, because R applies neither. The fused reduction order and the hand-written
 digamma move the last bits, so this mode is not bit-identical to eager, and Sec. 3.2 reports
 how far the two diverge on real data.

155 Covering a range of P needed one extra step. The eager path writes the $P \times P$ elimination
 as a Python loop over tensor slices, but Triton’s frontend admits no local arrays and no
 compile-time sequence building, so a register-resident matrix cannot be indexed by a loop
 variable: the algebra must appear as straight-line code specialized to each P . Because that
 grows quadratically, from 20 lines at $P=2$ to 174 at $P=6$, we emit it from a generator
 150 mirroring the eager elimination operation for operation rather than hand-writing each case;
 only the $P \in \{2, 4\}$ arms are hand-written. Both kinds of arm are checked against the eager
 Cox–Reid term at fixed α , which isolates the linear algebra from the optimizer’s path: over
 all covered P and $S \in \{8, 12, 24\}$ the log-posterior agrees to 3e-15 relative and its gradient
 to 3e-14, against committed test bounds of 1e-13 and 1e-12.

155 2.4 GLM acceleration

The GLM is implemented as a batched negative-binomial IRLS solve on the GPU, using one
 batched factorization across genes at each iteration. Genes that diverge or reach the DESeq2
 iteration cap use the same host-side fallback as the reference. Wald standard errors retain
 the $\mu \geq 0.5$ floor in `fitBeta.cpp`; LRTs use the full-reduced deviance difference.

160 2.5 LFC shrinkage acceleration

apeGLM (Zhu et al., 2019) is a per-gene optimization, so its L-BFGS state, line search, and
 convergence flags are batched on the GPU. We match the reference optimizer’s initialization
 and stopping rules because extreme, low-count genes can have competing posterior modes.

As in `lfcShrink`, only the shrunken coefficient and its standard error are replaced; p-values
 165 remain those from the Wald fit.

2.6 Validation methodology

Reference fixtures are generated by `scripts/generate_r_fixtures.R` against R DE-Seq2 1.52.0: three single-factor designs at 12×200 , 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim \text{batch} + \text{condition}$ with a three-level batch
 170 ($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$). Every intermediate is checked against R: size factors, `dispGeneEst`, `dispFit` (the parametric trend on every fixture, plus the "local" and "mean" trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β , μ , the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the apeGLM prior
 175 scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.2 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2).

3 Results

3.1 Experimental setup

180 All GPU measurements are taken on one NVIDIA A100-SXM4-40GB under PyTorch 2.6 with CUDA 12.4 and NVIDIA driver 550.90.07. The reference CPU baseline (host A) is R DESeq2 1.52.0 on a 12-core box, with R 4.6.0 and apeglm 1.34.0. Every timing is a warm median of N repetitions with `torch.cuda.synchronize()` on both sides of the timed region, so no GPU work is left in flight when the clock stops. End-to-end R timings use both one
 185 worker and 12 BiocParallel workers; Table 3 uses the faster configuration for each design.

We validate on three real, published RNA-seq datasets sliced into six designs spanning $P = 2$ –5 model terms: pasilla (Brooks et al., 2011) (*Drosophila*, $n=7$, one- and two-factor),

airway (Himes et al., 2014) (human, $n=8$, three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood vs. skeletal-muscle contrast (54,922 genes). The first two are distributed as Bioconductor experiment packages; the GTEx counts are recount2’s uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017), from which we draw 150 samples per tissue at a fixed seed and convert base coverage to read counts by dividing by the average read length. Scaling studies use fixed-seed negative-binomial simulations where the gene and sample axes can be swept independently.

3.2 Numerical parity with R

Correctness is checked per substep against R on all six real datasets (Table 2). Synthetic fixtures cover three single-factor designs, a multi-factor design, and a continuous covariate. On these fixtures, size factors agree to $<1e-10$, gene-wise dispersion to $\sim 1e-9$, and MAP dispersion, β , μ , the Hessian and Wald SEs to $\sim 1e-6$ relative. A further suite (tests/test_triton_dispersion.py) pins the fused kernel against the eager path once per covered P , the axis along which its specialized arms (Sec. 2.3) could diverge without any other test noticing.

Table 2: Per-substep agreement with R DESeq2 1.52.0 across all six real datasets. Each dataset is scored independently by the substep’s natural metric ($n=6$; the score is per-dataset, not a pooled statistic), and both the median and the least-favorable of the six are shown. The R reference here is the substep chain (estimateSizeFactors $\rightarrow$ estimateDispersions $\rightarrow$ DESeq $\rightarrow$ results). Because size factors and dispersions already exist, the DESeq call times the Wald fit and, where applicable, count-outlier replacement and refitting. Reproduce: bench/bench.py $\rightarrow$ bench/results/reference_parity.json.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $ \Delta $ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

All 30 substep checks pass, and every one of the six dispersion comparisons is within 0.44%

of R, the worst being `gtex_blood_muscle` at $4.4\text{e-}3$. The small-degrees-of-freedom case is not
205 the outlier one might expect. R enters the simulation-based prior-variance estimator only
when the residual degrees of freedom satisfy $0 < m - p \leq 3$, so `airway` ($n=8$, $P=5$, dof 3)
is the only one of the six datasets to take that branch at all; because the branch reproduces
R’s scalar bit for bit (Sec. 2.3), its dispersion agrees to $4.5\text{e-}4$, its raw LFC to $3.0\text{e-}5$, and its
significant-gene set matches R’s *exactly* (Jaccard 1.000). The apeGLM shrinkage matches
210 R to Pearson $r \geq 0.997$ on every dataset, with p95 $|\Delta| \leq 3.4\text{e-}3$. The residual is confined to
flat, near-degenerate genes on which two limited-memory-BFGS implementations break a tie
between equal-posterior optima differently; the shrunk LFC is a reported effect size and does
not feed the significance calculation, so it changes no call. Across the 25 synthetic dispersion
configurations of Sec. 3.4 and Sec. 3.5, the graph path reproduces eager exactly: gene-wise,
215 MAP and final dispersion all agree to $\max |\Delta| = 0$ (`benchmarks/results_a100*.json`).
Triton’s fused reduction order and hand-written `digamma` change the last bits; its dispersion
agrees with eager to $\leq 3.2\text{e-}7$ on the five small- n sets. The standard-pipeline `gtex` comparison
includes count replacement and refitting in every mode (Table 3).

3.2.1 Parity in the standard DE plots

220 The residuals above are summary statistics, but a DESeq2 run is read through a small set of
standard diagnostic plots, and parity is only useful if it survives into those. Fig. 3 redraws
the three an analyst looks at first (the dispersion-estimate, MA and volcano plots) from both
pipelines by identical code, on axis limits taken from R and reused for cuDESeq2 so the rows
are compared rather than each rescaled to its own data. They are visually indistinguishable:
225 the gene-wise MLE cloud, the fitted trend, the shrunken MAP band and the 141 genes the
outlier rule exempts from shrinkage all land in the same places, and the same 3,993 genes
are called at $\text{padj} < 0.05$.

Fig. 4 quantifies the same agreement across all six designs in the terms a DE analysis is
used for. The two pipelines put the same genes at the top of the ranked list: at every N up

airway (~ cell + dex, n=8, 33,469 genes, P=5)

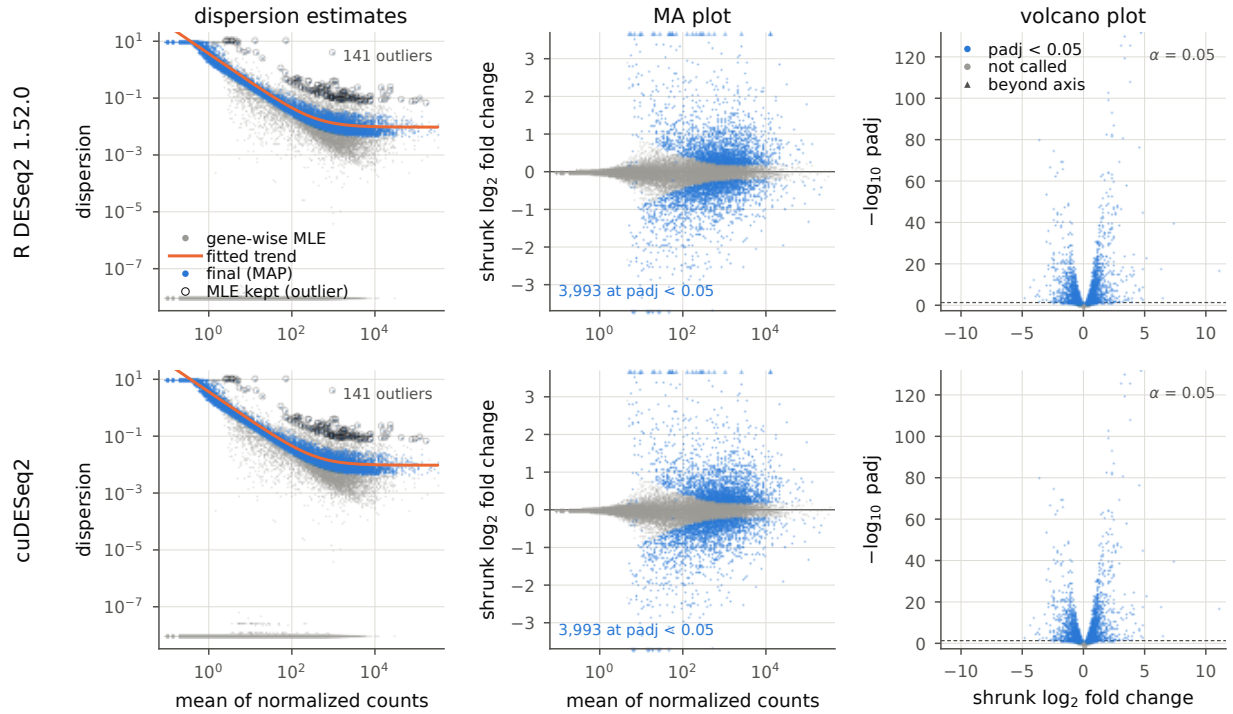


Figure 3: The three standard DESeq2 diagnostics on `airway` (~ cell + dex, $n=8$, $P=5$), drawn from R DESeq2 1.52.0 (top) and cuDESeq2 (bottom) by the same plotting code. *Left*, the `plotDispEsts` view: gene-wise MLE (grey), fitted trend (orange), final estimate (blue), and circled genes whose MLE exceeds the trend by more than two residual SDs and is therefore kept unshrunk. *Center*, MA plot of the apeGLM-shrunk LFC, with out-of-range genes on the boundary as in `plotMA`. *Right*, volcano plot. Axis limits are computed from R's output and reused for cuDESeq2. Reproduce: `validation/export_r_dispersion_details.R` then `validation/make_paper_figures.py`.

230 to 2,000 and on every design they share $\geq 99.6\%$ of the top N , and their called sets agree to a Jaccard index of ≥ 0.961 across the whole range $\alpha \in [10^{-3}, 0.2]$. At $\alpha=0.05$, four of the six agree exactly and the other two reach 0.99963 and 0.99997. Ranking uses $|\text{Wald statistic}|$ rather than the p-value because on **gtex** R reports $p = 0$ for hundreds of underflowed genes, so a p-value ranking would compare arbitrary tie-breaks among them.

235 The first is a μ floor in the Wald standard error. R forms the weights for its coefficient covariance from a μ already thresholded at `minmu (fitBeta.cpp)`, whereas our IRLS returns μ unthresholded because Cook’s distance requires it raw; omitting the floor when forming $X^T W X$ for the SE drops any sample with $\mu < 0.5$ out of the weight matrix and inflates the SE. The effect is invisible on ordinary genes and severe on degenerate ones. Without the
240 floor the SE runs 22% high on genes where one contrasted group is *entirely* zero, which is enough to change `airway_cell`’s called set and pull its Jaccard index against R’s 206 genes to 0.971, and $\sim 7\%$ high on near-zero-count genes whose dispersion is pinned at the ceiling, moving their raw p-values by up to 0.07. With the floor applied the SE matches R to 0.03% on those same degenerate genes, `airway_cell` and `airway` both reproduce R’s called set
245 *exactly*, and the histograms of panel (a) coincide. Only the few genes carrying $\mu < 0.5$ in some sample are affected at all, which is why every aggregate metric in Table 2 is essentially unchanged either way. A suite scoring only aggregates can miss a detail of this kind.

The standard wrapper has an additional branch when a design cell contains at least `minReplicatesForReplace= 7` samples. Both implementations flag counts above the 0.99
250 quantile of the Cook’s-distance F distribution, replace eligible counts with the 20%-trimmed mean of normalized counts rescaled by the sample size factor, and refit the affected genes. The refit re-estimates the gene-wise dispersion while retaining the original fitted trend and dispersion prior, then repeats the MAP and Wald fits. Original counts are retained for subsequent shrinkage. This branch is unreachable for the five seven- or eight-sample designs;
255 it is exercised by **gtex**, with 150 samples in each group. Table 2 therefore tests the same standard wrapper path for both implementations rather than comparing `cuDESeq2` with

nbinomWaldTest alone.

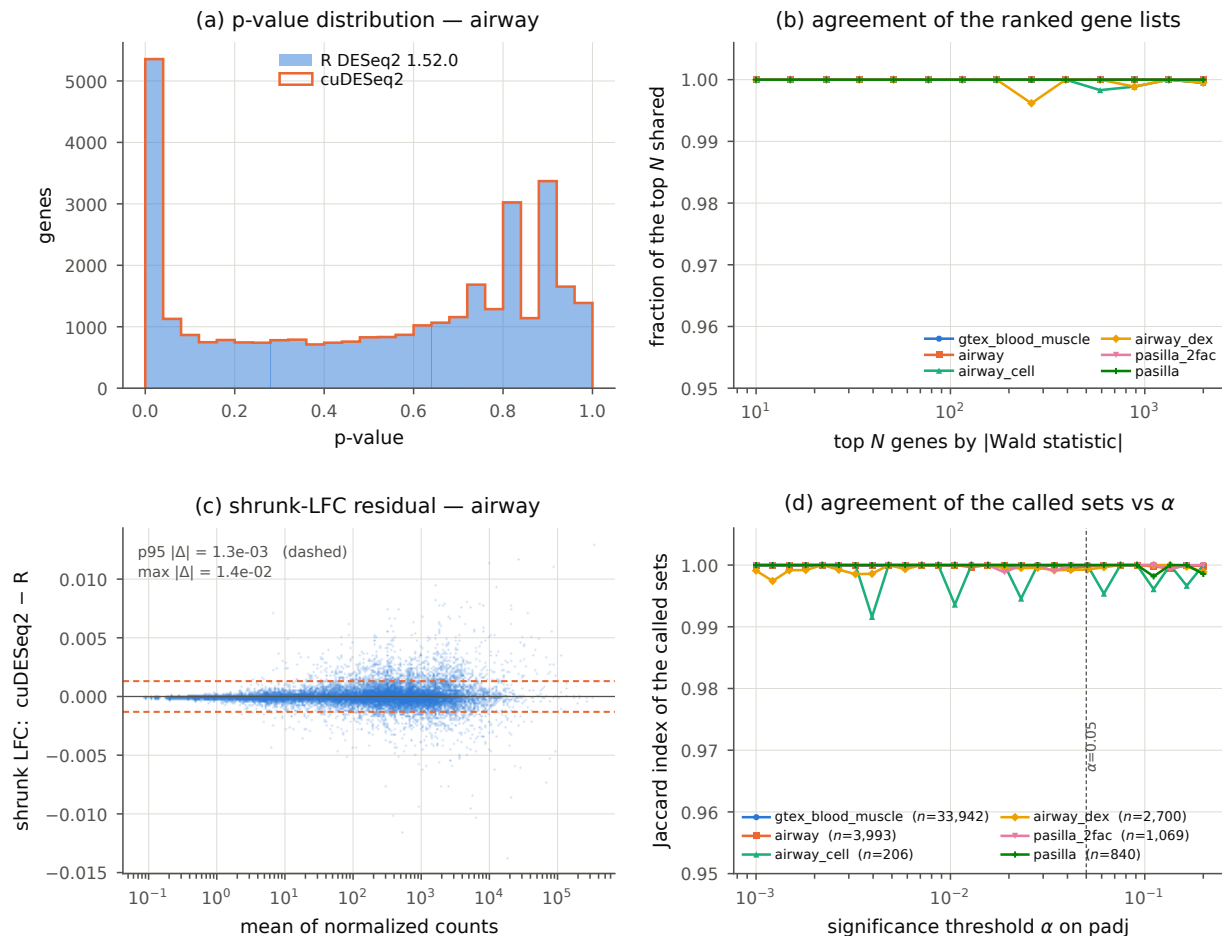


Figure 4: Quantitative agreement across all six designs. (a) p-value distributions on **airway**, overlaid. (b) Fraction of the top N genes shared by the two rankings, ranked by $|\text{Wald statistic}|$ to avoid comparing tie-breaks among genes whose p-value has underflowed in R. (c) apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. (d) Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at $\alpha=0.05$ in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The **gtex** curve now includes count replacement and refitting in both pipelines.

3.3 Real-data timing versus R

The graph and Triton modes change the dispersion optimizer. The remaining stages use the same code, except that a standard-pipeline outlier refit invokes the selected dispersion optimizer again. None of the five small designs enters that branch. Their downstream timing

Table 3: End-to-end wall time on the real datasets: R 4.6.0/DESeq2 1.52.0 with one or 12 BiocParallel workers, and cuDESeq2 on one A100. The R runs and GPU runs use the standard pipeline; the `gtex` row includes count replacement and affected-gene refitting. Reproduce: `bench/run_r_parallel.R → bench/results/r_parallel_a100_12worker.json`; retained GPU timings: `bench/results/timings.json`.

Dataset	P	n	R DESeq2 (s)		cuDESeq2, one A100 (ms)		
			1 worker	12 workers	eager	graph	Triton
pasilla	2	7	9.5	7.2	982	650	717
pasilla_2fac	3	7	10.1	7.4	1481	922	841
airway_dex	2	8	25.9	12.4	1307	968	898
airway_cell	4	8	29.4	10.9	4169	3579	3495
airway	5	8	34.7	12.0	2559	1681	1504
gtex	2	300	277.7	63.5	8510	8293	4754

The best-mode speedup over the faster R configuration is 3.1–13.8 $\times$; `airway_cell` and `airway_dex` are the endpoints.

differences are measurement noise or, for Triton, changes in the number of tolerance-based shrinkage iterations caused by last-bit dispersion differences.

Triton is the fastest total mode on five designs; graph is fastest on `pasilla`. Triton has
265 the fastest dispersion substep on all six: fusing the wider Cox–Reid solve (Sec. 2.3) cuts the dispersion substep by 8.9 $\times$ on `pasilla_2fac` ($P=3$) and 2.7 $\times$ on `airway` ($P=5$) against the eager loop (708 $\rightarrow$ 80 ms and 1586 $\rightarrow$ 581 ms, same run). `airway`’s dispersion substep also carries the ≈ 0.3 s CPU cost of the R-stream replay of Sec. 2.3, in every mode. All three
270 modes are nonetheless shipped, because what separates them is coverage and fidelity rather than speed: graph is bit-identical to eager at any P , whereas Triton is a GPU-only path limited to $P \leq 6$ and agrees with eager only to the last few bits (Sec. 3.2).

3.4 Scaling on synthetic data

Synthetic data lets the gene and sample axes move independently, which is what identifies
why the dispersion loop is slow rather than only how slow it is. We first establish the
275 bound. Hardware counters were unavailable on this instance, so the roofline is measured empirically: a three-array fp64 streaming probe sustains 1369 GB/s, or 88% of the A100’s

Table 4: Per-substep wall time (ms) on the real datasets, one-worker R on the reference host vs. the three cuDESeq2 modes. The `glm_fit` timing includes Cook’s-distance replacement and refitting. Reproduce: `bench/bench.py → bench/results/TABLES.md`.

Dataset	Substep	R	eager	graph	Triton
pasilla	dispersion	1722	402	71	20
	glm_fit	3878	38	37	37
	significance	206	63	63	63
	lfc_shrink	5143	478	478	596
	total	11091	982	650	717
pasilla_2fac	dispersion	2140	708	161	80
	glm_fit	3755	320	321	322
	significance	185	74	60	60
	lfc_shrink	5912	378	379	378
	total	12137	1481	922	841
airway_dex	dispersion	4707	429	109	51
	glm_fit	9924	55	53	53
	significance	1397	145	124	123
	lfc_shrink	14715	676	681	670
	total	30932	1307	968	898
airway_cell	dispersion	5794	748	186	60
	glm_fit	9584	37	36	36
	significance	362	114	115	115
	lfc_shrink	20157	3268	3240	3283
	total	36080	4169	3579	3495
airway	dispersion	8077	1586	713	581
	glm_fit	12686	62	60	60
	significance	872	117	117	117
	lfc_shrink	19712	793	790	746
	total	41570	2559	1681	1504
gtex	dispersion	188957	3500	3623	200
	glm_fit	239691	3147	2810	2709
	significance	496	623	623	621
	lfc_shrink	39330	1235	1233	1220
	total	471588	8510	8293	4754

normalization omitted (≤ 4 ms cuDESeq2 on every set); full versioned R records are in `bench/results/timings.json` and `bench/results/r_parallel_a100_12worker.json`. All GPU values come from the same A100 session.

1555 GB/s specified peak, and the loop’s kernels come nowhere near it. At $60 \times 20,000$ the elementwise weight update reaches 21% of sustained bandwidth and the `lgamma` and `digamma` evaluations 43% and 36%; at $60 \times 2,000$ all three fall to 3–8%, compute-bound on the special
 280 functions rather than starved for bandwidth. Neither regime is the binding constraint. What settles it is the assembled objective-and-gradient evaluation `_lp_and_dlp`, the body of the loop: 1287 μ s at 20,000 genes and 1292 μ s at 2,000. A tenfold change in arithmetic and bytes moved produces no change in wall time. This is the signature of a loop paying mainly for launches and dispatch. That is the cost graph replay and Triton fusion attack, and why they
 285 buy more than a better-tuned kernel could.

The synthetic scaling experiments below isolate the dispersion optimizer rather than report end-to-end DESeq2 speedups; they therefore do not exercise the standard replacement/refit wrapper branch. The real-data timings in Table 3 use that branch where applicable. If the loop is launch-bound, the accelerators should pay most where iterations are many
 290 and each is cheap, and fade as every iteration acquires enough arithmetic to fill the device. The sample axis tests that directly (Table 5). Holding genes at 2,000 and sweeping S from a 2v2 pilot to a 2,000-sample cohort, the graph’s speedup on the isolated gene-wise ascent loop declines from $7.6\times$ at $S=4$ to $0.96\times$ at $S=2000$: by two thousand samples the launch overhead it removes is a rounding error against the per-iteration work, and replay is no longer
 295 worth its bookkeeping. Triton decays over the same range but never loses, from $69.0\times$ to $7.7\times$, since fusion removes the intermediate traffic as well as the launches. Two features of the sweep are structural rather than noise. The $S=4$ and $S=6$ rows run the full 100-iteration cap while every larger S converges in 13–25, which is most of why the small-sample regime is both the slowest and the most accelerable. And $S=4$ is the one row where the stage-level
 300 speedup ($1.8\times$) falls far below the loop-level one ($69.0\times$): with $m - p = 2$ that design takes the stochastic KL branch (Sec. 2.3), whose compiled replay of R’s random stream is serial host work no GPU mode touches, contributing ≈ 470 ms to every mode at $S=4$ and nothing at $S=6$. It is Amdahl’s remainder for that row.

Table 5: Sample-axis scaling at 2,000 genes, `~condition` ($P=2$), single A100. “GW loop” is the isolated gene-wise Cox–Reid gradient-ascent fit (Sec. 2.3), the part the accelerators act on; “full stage” is `fit_dispersions` around it, including host-side trend fitting. Speedups are against eager in the same row. The graph is bit-identical to eager at every S ($\max|\Delta| = 0$); Triton is not, and agrees to $\leq 3.0\text{e-}8$ here. Reproduce: `benchmarks/bench_cuda_graph.py --sample-sweep → benchmarks/results_a100_samplesweep.json`.

S	iters	GW loop (ms)			GW speedup		full-stage speedup	
		eager	graph	Triton	graph	Triton	graph	Triton
4	100	350.2	46.1	5.1	7.59	68.99	1.61	1.81
6	100	335.2	46.7	5.1	7.18	66.05	6.20	40.35
30	13	44.9	10.8	3.1	4.15	14.29	3.81	13.70
60	15	51.5	11.4	3.2	4.51	16.11	3.50	14.10
200	16	55.3	14.7	3.2	3.77	17.43	3.10	13.92
1000	18	63.9	44.0	8.0	1.45	8.02	1.30	8.17
2000	25	128.3	133.5	16.6	0.96	7.71	1.08	6.88

The gene axis says the same thing from the other side. Taking 2,000 to 20,000 genes
 305 at fixed $S=60$ multiplies the work tenfold but the eager stage time only by $1.7\times$ at $P=2$
 (127→210 ms) and $1.2\times$ at $P=4$ (193→224 ms): nine tenths of the added genes are absorbed
 into launch latency the small case already paid. Because that fixed overhead is amortized
 over more genes at 20,000, it is also where the graph has least left to remove, its stage speedup
 dropping from $4.4\times$ to $1.7\times$ at $P=2$ ($3.7\times$ to $2.2\times$ at $P=4$). Graph capture is a warm-path
 310 optimization: the one-time capture costs 487–796 ms across these four configurations, so it
 is recovered only when many matrices of one shape are fitted in a session, as in the serving
 pattern the virtual-cell evaluation loops of the Introduction generate.

3.5 Ablations

Three design choices in Sec. 2.3–2.5 could reasonably have gone the other way, and each is
 315 worth separating from the headline numbers.

Chunk length, and why a full-length capture fails. The obvious way to graph
 an iterative loop is to capture all of it, and that is the worst configuration we measured.
 Because a captured graph cannot branch on a convergence flag, a capture of k iterations

must be replayed $\lceil \text{iters}/k \rceil$ times, so the loop executes $\lceil \text{iters}/k \rceil \cdot k$ iterations rather than
 320 the iters eager early-exits after; done genes are frozen no-ops but still cost their share of
 the kernel. Sweeping k over the divisors of $\text{maxit} = 100$ (Table 6) traces that cost model.
 The full-length capture $k=100$ rounds 26 real iterations up to 100 and gives back almost all
 the benefit: $1.83\times$ instead of $6.01\times$ at 2,000 genes, $0.99\times$ instead of $2.42\times$ at 20,000, where
 it is indistinguishable from not graphing at all. What the sweep shows is that the padded
 325 iteration count, not the replay count, sets the time: rows executing the same number of
 iterations land within a couple of per cent of each other however many replays that took
 ($k=25$ and $k=50$ both pad to 50 and differ by 0.4% at 2,000 genes; $k=2$ through $k=20$ all
 pad to 40 and differ by 2% at 20,000). At 2,000 genes $k=2$ is fastest ($6.01\times$) because it
 avoids padding, whereas at 20,000 genes $k=10$ and $k=20$ tie at $2.42\times$. The current $k=10$
 330 default is therefore a conservative compromise rather than a universal optimum.

Table 6: `GRAPH_CHUNK` sweep on the isolated gene-wise loop, single A100, `~condition`. “iters run” is $\lceil \text{iters}/k \rceil \cdot k$, the padded count the replay actually executes; the eager loop early-exits after 26 and 40 iterations respectively. $k=100$ captures the whole loop in one graph and is therefore the fixed-full-length-capture ablation. Every k is bit-identical to eager ($\max|\Delta| = 0$), so these are pure scheduling trade-offs. Reproduce: `benchmarks/bench_cuda_graph.py --sweep` $\rightarrow$ `benchmarks/results_a100_sweep.json`.

k	60 $\times$ 2,000 (eager 89.2 ms)			60 $\times$ 20,000 (eager 138.8 ms)		
	iters run	graph (ms)	speedup	iters run	graph (ms)	speedup
2	26	14.83	6.01	40	58.24	2.38
5	30	16.28	5.48	40	57.48	2.41
10	30	16.18	5.51	40	57.29	2.42
20	40	20.73	4.30	40	57.45	2.42
25	50	25.42	3.51	50	71.15	1.95
50	50	25.34	3.52	50	70.96	1.96
100	100	48.73	1.83	100	139.66	0.99

Per-gene early exit in the fused kernel. The Triton kernel’s loop is guarded per
 program by `(it < maxit) & (~done)` (Sec. 2.3) rather than run for a fixed trip count, and
 the iteration counts of Table 5 bound what that is worth: outside the small-sample regime
 the loop converges in 13–25 iterations against a cap of 100, so a fixed-trip kernel would

335 execute four to nearly eight times the loop body for an identical answer. The saving is largest exactly where the batch is widest, since a fixed trip count makes every gene wait for the cap rather than for its own convergence. This is a bound inferred from the shipped kernel’s measured iteration counts rather than a measured A/B: the released code exposes no fixed-trip switch to compare against.

340 **The apeGLM optimizer is a parity choice, not a speed choice.** A batched Newton solver for the shrinkage step is both simpler to write and faster than porting apeGLM’s L-BFGS, and it disagrees with R. The reason is the one described in Sec. 2.5: the Cauchy-prior posterior is bimodal for extreme-effect genes, R reports whichever mode its own optimizer reaches from its own initialization, and a solver that finds the global optimum therefore
345 returns a better estimate that is not the reference’s. This is the one place where improving on the reference and reproducing it are in direct conflict, and a drop-in replacement has to choose reproduction. Porting the reference’s optimizer restores agreement to $r \geq 0.997$ on shrunk log-fold-changes across all six designs (Table 2) while keeping the stage on the GPU with no per-gene host fallback.

350 4 Discussion

cuDESeq2 shows that the standard DESeq2 analysis pipeline can be moved to a GPU without replacing its statistical model or sacrificing quality. Across six real-data designs, the implementation reproduced the intermediate quantities used by normalization, dispersion estimation, GLM fitting, significance testing, and LFC shrinkage (Table 2), while reducing
355 end-to-end runtime by 3.1–13.8× relative to the fastest R DESeq2 run on the A100 benchmark host. These gains make repeated, reference-compatible differential-expression analyses more practical in settings such as model evaluation, where the same analysis is run many times.

The primary speedup comes from replacing sequential per-gene optimization with batched

GPU computation. We obtain further speedup through CUDA-graph replay and Triton fusion which reduce the overhead of the dispersion iterations. Speedup varies across designs due to factors such as: the time spent in dispersion fitting, shrinkage, and the remaining CPU steps depends on sample size and the design matrix.

We note that our validated scope includes single-factor designs with up to four levels, additive multi-factor designs with up to $P=5$, and continuous covariates. The fused Triton path supports $P \in \{2, \dots, 6\}$ and otherwise uses the eager implementation.

We note that in many instances numerical equivalence depends on subtle implementation details beyond published technical specifications. The dispersion prior, Wald standard errors, and apeGLM shrinkage each exposed behavior that is not fixed by the model specification alone (Sections 2.3, 3.2.1, and 2.5). Reproducing such details is sometimes necessary to obtain DESeq2 equivalent output. For example, we applied DESeq2’s minmu floor when forming the GLM weight matrix used to calculate Wald standard errors; without it, genes with near-zero fitted counts receive different standard errors and can produce different p-values.

In other instances we achieve fidelity to the `deseq2` method description with minor differences in floating-point rounding. For example Triton evaluates the same dispersion objective in a fused kernel, but its different floating-point reduction order and digamma implementation introduce small numerical differences from eager PyTorch. On GTEx, the dispersion difference was 1.3×10^{-4} at the 95th percentile and at most 1.69×10^{-2} for any gene; across the five smaller designs, it was at most 3.2×10^{-7} . In such instances, it is worth noting that these differences arise from implementation choices, not from a change to the DESeq2 model, and neither implementation is inherently closer to the intended calculation under that statistical model specification.

In conclusion, `cuDESeq2` preserves the DESeq2 analysis while making repeated differential-expression analyses substantially faster on a GPU. Our hope is that this improvement will support the bioinformatics community by enabling reference-compatible

DE analysis in workflows that require many repeated comparisons.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

The pasilla benchmark uses the *Drosophila melanogaster* RNA-seq experiment distributed in the Bioconductor `pasilla` data package (<https://bioconductor.org/packages/pasilla>) (Brooks et al., 2011). The airway benchmark uses the human airway smooth-muscle-cell experiment in the Bioconductor `airway` data package (<https://bioconductor.org/packages/airway>) (Himes et al., 2014). The GTEx whole-blood versus skeletal-muscle benchmark uses recount2 gene-level data for SRA study SRP012682 (https://recount-omni.s3.amazonaws.com/recount2/v2/SRP012682/rse_gene.Rdata) (GTEx Consortium, 2020; Collado-Torres et al., 2017).

Source code, benchmark inputs, reference-generation scripts, and the Docker environment are available at https://github.com/synthesizebio/gpu_deseq under the MIT license.

References

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma-poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory

map between *Drosophila* and mammals. *Genome Research*, 21(2):193–202, 2011. doi:
10.1101/gr.108662.110.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

GTEX Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014. doi: 10.1371/journal.pone.0099625.

Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9):btad547, 2023. doi: 10.1093/bioinformatics/btad547.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

435

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary algorithms

440 Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 FUSED TRITON DISPERSION FIT (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.

Ensure: optimized log dispersions $\mathbf{a}$.

- 1: **Launch** one Triton program for every gene g in parallel.
 - 2: Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
 - 3: Initialize $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
 - 4: **repeat** inside the same fused program
 - 5: Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
 - 6: If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
 - 7: Compute the gradient $q \leftarrow \partial \ell / \partial a$.
 - 8: Propose the gradient-ascent step $\tilde{a} \leftarrow a + \kappa q$.
 - 9: Accept or reject $\tilde{a}$ using the Armijo condition; adapt κ .
 - 10: **until** converged or *maxit* is reached.
 - 11: Store the final a_g once in global memory.
-

Supplementary Algorithm S2 DETAILED FUSED TRITON DISPERSION KERNEL (CR-MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$, $P \in \{2, \dots, 6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^G$; compile-time $h \equiv \text{HASPRIOR}$; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0 .

Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

```

1: parallel for Triton program  $g \in \{0, \dots, G-1\}$  do (one program per gene)
2:   Load  $\mathbf{y}_g$ ,  $\boldsymbol{\mu}_g$ , and  $\mathbf{X}$  once into program-local state.
3:   Set  $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ ,  $\kappa \leftarrow \kappa_0$ ,  $n_{\text{acc}} \leftarrow 0$ , and  $c \leftarrow 0$ .
4:   Set  $(\ell, q) \leftarrow \text{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
5:   while  $c < \text{maxit}$  and not converged do
6:      $c \leftarrow c + 1$  and propose  $\tilde{a} \leftarrow a + \kappa q$ .
7:     If  $q \neq 0$  and  $\tilde{a} \notin [-30, 10]$ , shorten  $\kappa$  so that  $\tilde{a}$  lies on the violated boundary.
8:     Set  $(\tilde{\ell}, -) \leftarrow \text{ObjectiveGradient}(\tilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
9:     Set  $r \leftarrow [\tilde{\ell} \geq \ell + \varepsilon \kappa q^2]$ . (Armijo acceptance flag)
10:    Select  $a' \leftarrow r ? \tilde{a} : a$ ,  $\ell' \leftarrow r ? \tilde{\ell} : \ell$ , and  $\Delta \leftarrow \ell' - \ell$ .
11:    Set  $(-, q') \leftarrow \text{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$  and  $q \leftarrow r ? q' : q$ .
12:    Set  $n'_{\text{acc}} \leftarrow n_{\text{acc}} + r$  and  $\kappa_{\text{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$ .
13:    If  $r$  and  $n'_{\text{acc}} \bmod 5 = 0$ , set  $\kappa_{\text{acc}} \leftarrow \kappa_{\text{acc}}/2$ .
14:    Set  $\kappa \leftarrow r ? \kappa_{\text{acc}} : \kappa/2$ ,  $n_{\text{acc}} \leftarrow n'_{\text{acc}}$ ,  $a \leftarrow a'$ , and  $\ell \leftarrow \ell'$ .
15:    Converge if  $r \wedge (\Delta < 10^{-6} \vee a < \log(\text{minDisp}/10))$ .
16:  end while
17:  Store  $a_g \leftarrow a$  and  $c_g \leftarrow c$  in global memory.
18: end parallel for
19: function ObjectiveGradient( $a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2$ )
20:   Set  $\alpha \leftarrow e^a$ ,  $\mathbf{W} \leftarrow \text{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$ , and  $\mathbf{B} \leftarrow \mathbf{X}^\top \mathbf{W} \mathbf{X}$ .
21:   Compute  $\ell \leftarrow \ell_{\text{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2} \log \det(\mathbf{B})$ .
22:   Compute  $q \leftarrow \partial \ell / \partial a$ , including  $\text{tr}(\mathbf{B}^{-1} \partial \mathbf{B} / \partial \alpha)$  via an unrolled  $2 \times 2$  formula ( $P=2$ ) or  $P \times P$  LU solve.
23:   if  $h = 1$  then  $\ell \leftarrow \ell - (a - m)^2 / (2\sigma^2)$  and  $q \leftarrow q - (a - m) / \sigma^2$ . (MAP only)
24:   return  $(\ell, q)$ .
25: end function

```
